Dossier Técnico

Evolución Arquitectónica y Reactividad con Angular Signals

Módulo: Desarrollo de Interfaces (0488) | **Ciclo:** DAM (Grado Superior)
Alineación Curricular (Comunidad de Madrid):

- **RA 1:** Confecciona interfaces de usuario analizando y aplicando las arquitecturas y patrones de diseño.
  - *CE 1.a:* Se han identificado las diferentes arquitecturas de diseño de interfaces de usuario.
  - *CE 1.b:* Se han evaluado los patrones de diseño en la confección de interfaces.
- **RA 3:** Crea componentes visuales, gestionando eventos y desarrollando controles personalizados.
  - *CE 3.d:* Se han asociado acciones a los eventos.
  - *CE 3.e:* Se han probado y validado los componentes creados.

1. Base Teórica Técnica: El Paradigma de las Señales (Signals)

1.1. Contexto Histórico: ¿Qué se hacía antes (Angular <= 15)?

Hasta la llegada de las Signals, Angular gestionaba el estado y la reactividad basándose en dos pilares que, aunque potentes, presentaban problemas de rendimiento y curva de aprendizaje a escala:

1. **Zone.js y el "Dirty Checking":** Angular delegaba la detección de cambios a Zone.js, una librería que "parcheaba" las APIs asíncronas del navegador (clicks, setTimeout, llamadas HTTP). Cuando ocurría un evento, Zone.js notificaba a Angular, el cual recorría **todo el árbol de componentes** (de arriba a abajo) para comprobar si algún valor había cambiado. Esto era ineficiente en aplicaciones complejas (como las híbridas en Ionic).
2. **RxJS para Estado Síncrono:** Para evitar el chequeo global, los desarrolladores Senior implementábamos la estrategia OnPush junto con BehaviorSubject (RxJS) y el pipe async. El problema es que RxJS está diseñado para flujos de eventos asíncronos (cadenas de tiempo), no para almacenar estado síncrono. Esto generaba sobreingeniería, fugas de memoria si no se desuscribían los observables, y una curva de aprendizaje excesivamente pronunciada para los alumnos.

1.2. El Nuevo Estándar (Angular 22): Angular Signals

Una Señal (Signal) es un contenedor alrededor de un valor que notifica a sus consumidores (templates, funciones u otras señales) cuándo dicho valor cambia.

Ventajas clave frente al modelo anterior:

- **Reactividad Granular (Fine-grained reactivity):** Angular ya no necesita Zone.js ni recorrer el árbol de componentes. Sabe *exactamente* qué nodo del DOM depende de qué señal, actualizando única y exclusivamente ese fragmento. Esto es vital para el rendimiento en dispositivos móviles mediante Capacitor.
- **Lectura Síncrona:** A diferencia de RxJS, el valor de una señal siempre está disponible de forma síncrona mediante miSenal().
- **Arquitectura "Zoneless":** En Angular 22, las aplicaciones pueden prescindir totalmente de Zone.js, reduciendo el tamaño del *bundle* y mejorando el tiempo de carga.

Tipos de Signals:

1. **signal() (WritableSignal):** Contiene un valor que puede ser modificado mediante .set() (sobrescribir) o .update() (modificar basándose en el valor anterior).
2. **computed():** Una señal de solo lectura que deriva su valor de otras señales. Se actualiza de forma "perezosa" (lazy) y se memoriza (cachea) hasta que sus dependencias cambien.
3. **effect():** Una operación secundaria (side-effect) que se ejecuta automáticamente cuando cambian las señales que lee en su interior (ej. guardar en localStorage o depurar).

2. Ejemplo de Código Funcional Paso a Paso

A continuación, implementaremos un componente de "Carrito de Compras" para Ionic, demostrando el uso de signal, computed y effect.

```
TypeScript
import { Component, computed, effect, signal, ChangeDetectionStrategy } from
'@angular/core';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonLabel, IonButton,
IonIcon, IonBadge } from '@ionic/angular/standalone';

// Interfaz para el tipado de datos
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
}

@Component({
  selector: 'app-carrito-signals',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush, // Optimización crítica
  imports: [IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonLabel, IonButton,
IonIcon, IonBadge],
  template: `
    <ion-header>
      <ion-toolbar color="primary">
        <ion-title>Carrito Reactivo</ion-title>
        <!-- Lectura de la señal computada en el template usando () -->
        <ion-badge slot="end" color="tertiary" class="ion-margin-end">
          Total: {{ precioTotal() }}€
        </ion-badge>
      </ion-toolbar>
    </ion-header>

    <ion-content>
      <ion-list>
        <!-- Nuevo Control Flow de Angular -->
        @for (prod of productos(); track prod.id) {
          <ion-item>
            <ion-label>
              <h2>{{ prod.nombre }}</h2>
              <p>{{ prod.precio }}€</p>
            </ion-label>
```

```
          <ion-button fill="clear" (click)="eliminarProducto(prod.id)">
            Eliminar
          </ion-button>
        </ion-item>
      } @empty {
        <ion-item>
          <ion-label class="ion-text-center">El carrito está vacío</ion-label>
        </ion-item>
      }
    </ion-list>

    <div class="ion-padding">
      <ion-button expand="block" (click)="agregarProductoAleatorio()">
        Añadir Producto Random
      </ion-button>
    </div>
  </ion-content>
  `
})
export class CarritoSignalsComponent {
  // 1. Estado Writable (WritableSignal)
  public productos = signal<Producto[]>([
    { id: 1, nombre: 'Licencia IDE', precio: 89 },
    { id: 2, nombre: 'Teclado Mecánico', precio: 120 }
  ]);

  // 2. Estado Derivado (Computed Signal) - Se recalcula solo si 'productos' cambia
  public precioTotal = computed(() => {
    return this.productos().reduce((total, prod) => total + prod.precio, 0);
  });

  constructor() {
    // 3. Efecto Secundario (Effect) - Se ejecuta en la inicialización y cada vez que 'productos' cambie
    effect(() => {
      console.log(`[Persistencia] Estado actual del carrito guardado. Items:
${this.productos().length}`);
      // Aquí integraríamos Preferences API de Capacitor para persistir localmente
    });
  }

  // Métodos de mutación de estado
  public agregarProductoAleatorio(): void {
    const nuevoId = Math.floor(Math.random() * 1000) + 3;
    const nuevoProducto: Producto = { id: nuevoId, nombre: `Monitor ${nuevoId}`, precio: 250 };
```

```
    // Mutación usando .update() - Inmutabilidad recomendada
    this.productos.update(estadoAnterior => [...estadoAnterior, nuevoProducto]);
  }

  public eliminarProducto(id: number): void {
    this.productos.update(estadoAnterior => estadoAnterior.filter(p => p.id !== id));
  }
}
```

Análisis Técnico Arquitectónico

Implementación de Signals en Angular 22

1. Detección de Cambios y Rendimiento (Change Detection)

```
@Component({
  ...
  changeDetection: ChangeDetectionStrategy.OnPush,
})
```

El uso de OnPush es obligatorio en arquitecturas reactivas modernas. Por defecto, Angular (con Zone.js) utiliza la estrategia Default, evaluando las expresiones del template en cada evento asíncrono del navegador.
Al establecer OnPush, le indicamos al framework que **solo evalúe este componente si**:

1. Recibe nuevas referencias en sus decoradores @Input().
2. Se emite un evento desde el propio componente (ej. un (click)).
3. **Una** Signal **vinculada a su template emite un nuevo valor.**

Aquí radica la potencia de Angular 22: las Signals informan directamente a la vista, permitiendo un renderizado granular ("Zoneless") sin necesidad de comprobar el árbol de componentes completo.

2. Grafo de Dependencias Reactivo: WritableSignal

```
public productos = signal<Producto[]>([ ... ]);
```

La función signal() crea un **nodo productor** (Producer) dentro del grafo de reactividad de Angular. Devuelve una WritableSignal. Internamente, Angular almacena el valor y mantiene un registro de los "consumidores" (Consumers) que leen esta señal (como el template o las señales computadas).

```
this.productos.update(estadoAnterior => [...estadoAnterior, nuevoProducto]);
```

En el método agregarProductoAleatorio(), se utiliza .update() en lugar de .set().
Debemos utilizar el operador de propagación (Spread Operator ...) o métodos que devuelvan una **nueva referencia en memoria** (como .filter() en la eliminación). Si mutáramos el array original con push(), la referencia de memoria (el puntero) no cambiaría, y Angular, bajo la estrategia OnPush, no notificaría al DOM de la necesidad de repintar la interfaz, provocando una desincronización entre el estado y la vista.

3. Evaluación Perezosa (Lazy Evaluation) con computed()

```
public precioTotal = computed(() => {
  return this.productos().reduce((total, prod) => total + prod.precio, 0);
});
```

computed() crea un nodo que es **tanto consumidor como productor**.

1. **Consumidor:** Lee la señal this.productos(). Automáticamente, Angular registra que precioTotal depende de productos.
2. **Productor:** El template consume precioTotal() en el <ion-badge>.

Diferencia crítica con funciones clásicas:
En versiones anteriores de Angular, llamar a una función {{ calcularTotal() }} en el HTML provocaba que la lógica se ejecutara decenas de veces por segundo en cada ciclo de detección.
Con computed(), el valor devuelto se **memoriza (caché)**. Angular no volverá a ejecutar el .reduce() a menos que detecte que el puntero de memoria de la señal subyacente (this.productos) ha cambiado. Esto ahorra ciclos de CPU fundamentales en dispositivos móviles (RA 1).

4. Intercepción de Efectos Secundarios (effect())

```
effect(() => {
  console.log(`... Items: ${this.productos().length}`);
});
```

La función effect() es un nodo **exclusivamente consumidor**. Se utiliza para ejecutar operaciones que no modifican el estado de la UI directamente, sino que interactúan con APIs externas (localStorage, IndexedDB, llamadas a Capacitor, manipulación directa del DOM, etc.).

Comportamiento interno:

- Se ejecuta asíncronamente al menos una vez (fase inicial).
- Durante esa primera ejecución, registra cualquier señal que se lea en su interior (this.productos()).
- A partir de ahí, solo se disparará de nuevo si esas señales específicas cambian.
- **Gestión de memoria:** Al declararse en el contexto de inyección (dentro del constructor), Angular vincula la vida de este efecto a la del componente. Cuando el componente se destruye (ngOnDestroy), el efecto se cancela automáticamente, eliminando el riesgo de *memory leaks* (fugas de memoria) típicas de las suscripciones manuales de RxJS no controladas.

5. Renderizado Optimizado con el Nuevo Control Flow (@for)

@for (prod of productos(); track prod.id) { ... }

El nuevo bloque @for reemplaza a la directiva estructural *ngFor. Su compilación es nativa, lo que reduce el sobrecoste en tiempo de ejecución.
El parámetro track prod.id **es obligatorio**. Implementa un algoritmo de reconciliación que permite a Angular identificar unívocamente cada elemento del DOM iterado. Si se elimina el producto con id: 2, Angular no destruye ni vuelve a crear toda la lista de <ion-item>; gracias a track, localiza exactamente el nodo del DOM correspondiente al id: 2 y lo elimina quirúrgicamente (DOM patching), reduciendo el reflow y repaint del navegador al mínimo absoluto (CE 3.d).